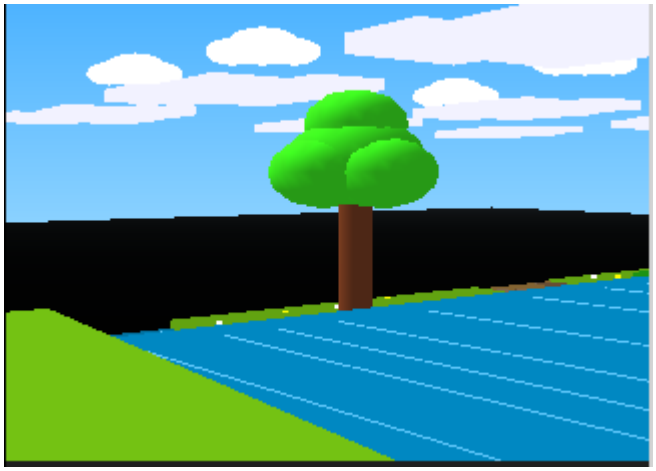
INTELIGENCIA ARTIFICIAL

E.T.S. de Ingenierías Informática y de Telecomunicación

Tutorial: Práctica 2 (Parte 1)

Agentes Reactivos/Deliberativos



DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN E INTELIGENCIA ARTIFICIAL

UNIVERSIDAD DE GRANADA

Curso 2025-2026

1. Introducción

El objetivo de la práctica es definir un comportamiento reactivo/deliberativo para un par de agentes que tienen como misión reconocer y desplazarse por un mundo bidimensional cuadriculado. En este tutorial se pretende introducir al estudiante en la dinámica que debería seguir para la construcción de su práctica y para ellos se resolverá casi completamente el nivel 0 de dicha práctica.

En la práctica se pueden considerar tres partes claramente diferenciadas. En los niveles 0 y 1 el estudiante tiene que trabajar con comportamientos reactivos, mientras que en los niveles 2, 3 y 4 trabajará con comportamiento deliberativos. En la última parte de esta práctica aparecerán procesos mixtos que deben combinar comportamientos reactivos y deliberativos. En este tutorial nos centraremos en los niveles reactivos.

El objetivo de la primera parte de la práctica es definir comportamientos reactivos para agentes que tienen capacidad de moverse sobre un terreno y cuya misión es descubrir los elementos que forman parte de ese terreno que le rodea. Aconsejamos que la construcción de dicho comportamiento se realice de forma incremental y mediante refinamiento paulatino. Si seguís este consejo, el proceso de construcción tendría la siguiente secuencia de subobjetivos:

1. Establecer una movilidad básica para el agente que le permita navegar por el mundo, inicialmente manteniéndose móvil y posteriormente ir mejorando su capacidad para descubrir zonas aún no exploradas.
2. Utilizar la matriz **mapaResultado** y **mapaCotas** para ir almacenando lo que voy viendo e ir cartografiando el terreno por el que el agente se va desplazando.
3. Definir qué comportamiento debe tener el agente frente a las otras entidades móviles del mundo.
4. Refinamiento del modelo general, para mejorar la capacidad de exploración del agente.

Este tutorial intenta ser una ayuda para poner en marcha la práctica y se centra en una aproximación inicial del subobjetivo 1 antes mencionado, es decir, dar los primeros pasos sobre el mundo.
Antes de nada, y para que el comportamiento que se defina esté más o menos estructurado y sea fácil de seguir, depurar y mejorar, es necesario entender que en el método que define el proceso (en nuestro caso, el método **think**[1]) se pueden considerar 2 fases:

(a) una primera fase de observación donde se actualizan los cambios que la última acción realizada provocó sobre el mundo
(b) una segunda fase para determinar la siguiente acción a realizar.

La segunda fase es realmente el comportamiento, pero la primera podría ser muy importante, ya que es necesaria para garantizar que la información usada para la toma de la decisión es correcta. La

[1]Para aquellos que acepten la idea de construir cada nivel en un módulo independiente como se propone en el software que se proporciona, sería la estructura de cada uno de esos módulos.

segunda fase está compuesta (en nuestro caso) por todas las reglas que guiarán el comportamiento del agente y que tendrán una estructura del tipo:

Regla 1:	**if (condicion^{1}){**
	}
Regla 2:	**else if (condicion^{2}){**
	}
.....	**.....**
.	
Regla n-1	**else if (condicion^{n-1}){**
	}
Regla por defecto	**else{**
	}

donde **condicion^{i}** es una condición lógica formada por los ***sensores*** o por las ***variables de estado*** definidas.

Sabemos que los sensores son los valores que recibimos directamente del mundo que nos rodea y cambian en cada iteración de la simulación y por consiguiente solo tienen vigencia en una iteración. Si queremos que nuestro agente tome una decisión con base en algo que no se está percibiendo en este instante, pero que sí percibió en algún instante anterior, tenemos que hacer uso de las ***variables de estado***. Las variables de estado son la memoria que tendrá el agente.

Sabemos que las variables de estado se usan formando parte de alguna o algunas condiciones de las reglas que compondrán el comportamiento del agente, ¿pero cómo se declaran?

En esta práctica en concreto la forma de trabajar con una variable de estado sería la siguiente:

- En el fichero "***.hpp***" del agente correspondiente, debajo de la parte que pone "***private:***" se declaran tantas variables como sean necesarias en la forma ***<tipo> <identificador>;***

```
Action last_action;

bool tiene_zapatillas;

int giro45Izq;
```

- En ese mismo fichero, en el constructor de la clase se inicializan. Hay dos constructores. El constructor que tiene como parámetro ***unsigned int size*** es el que usará para los niveles 0, 1 y 6. El constructor que tiene dos parámetros de tipo ***vector<vector<unsigned char>>*** es el que se usa en los niveles 2, 3, 4 y 5. En este tutorial trabajaremos sobre el nivel 0, por eso, usaremos el constructor que le corresponde.

```
last_action = IDLE;

tiene_zapatillas = false;

giro45Izq = 0;
```

- Ya en el fichero ".cpp", en el método correspondiente a la definición del comportamiento, en la parte dedicada a la actualización del mundo, se modifican los valores de las variables de estado en función de la acción realizada.

```
if (sensores.superficie[0] == 'D') {
        tiene_zapatillas = true;
}
```

- En la segunda parte de ese módulo se usa para determinar la siguiente acción a realizar.

```
  if (giro45Izq != 0){
        action = TURN_SR;
         giro45Izq --;
  }
    ……................
last_action = accion;
```

Con esta primera información básica, ya podemos empezar a construir los primeros comportamientos básicos de alguno de los agentes.

2. Los primeros pasos

Vamos a abordar el primero de los subobjetivos que hemos definido en la introducción "*movilidad básica*" y hacer que nuestro agente se mueva y lo vamos a hacer siguiendo las consideraciones que nos propone el nivel 0 de la práctica que consiste en moverse exclusivamente por casillas de tipo `camino 'C'`.

El comportamiento consistirá en mirar las casillas que quedan justo delante en el cono de visión del agente `Rescatador` y avanzar a la casilla de entre esas que sea de tipo camino. Cómo es posible que haya varias de tipo camino, establecemos como orden de prioridad que primero queremos avanzar a la que está justo delante (coordenada 2 del vector de visión), después la que está un poco a la izquierda (coordenada 1 del vector de visión) y después la que está un poco a la derecha (coordenada 3).
Una descripción en lenguaje natural de lo que nos gustaría ver en la evolución del agente sería que:

1. Si existe una casilla camino delante, ejecutará la acción `WALK` y pasará a ocupar dicha casilla.
2. Si es la casilla de un paso adelante, un paso a la derecha (coordenada 3 del vector de visión) la única que tuviera una casilla de tipo camino, esperaríamos que el agente hiciera en un primer instante la acción `TURN_SR`. De esta forma, en el instante siguiente estaríamos en la situación anterior.
3. Si es la de la izquierda (índice 1 del vector de visión) debería orientarse hacía dicha casilla, y aplicar la acción `TURN_SL`.
4. Si no se da ninguna de las anteriores, ejecutar la acción `TURN_SL` esperando que cambiando de orientación se encuentren nuevas casillas de tipo camino.

Definiremos la variable de estado `tiene_zapatillas` para almacenar si pasó por una casilla de tipo zapatillas y también una variable de estado que almacenará la última acción que realizó el agente. La forma en la que se realiza este proceso, ya se describió en el apartado anterior y el fichero "**ingeniero.hpp**" debería quedar se muestra a continuación:

```
174     /**
175     * @brief Convierte un plan de tubería en la lista de casillas usada
176     *        por el sistema de visualización.
177     * @param st    Estado de partida (no utilizado directamente).
178     * @param plan  Lista de pasos del plan de tubería.
179     */
180      void VisualizaRedTuberias(const list<Paso> &plan);
181
182
183
184    private:
185      // ==========================================================================
186      // VARIABLES DE ESTADO (PUEDEN SER EXTENDIDAS POR EL ALUMNO)
187      // ==========================================================================
188
189    Action last_action;       // Almacena la última acción ejecutada
190    bool tiene_zapatillas;    // Indica si el agente tiene las zapatillas
191    int giro45Izq;            // Indica el número de giros a la izq que quedan por dar.
192
193    };
194
195    #endif
```

Pasamos a definir el comportamiento que indicamos antes en el fichero "**ingeniero.cpp**", y lo haremos en el método **`ComportamientoRescatadorNivel_0`**.

Más abajo podréis encontrar la implementación de la primera versión del comportamiento que sigue caminos y en ella podemos distinguir las dos partes que ya se indicaban en la introducción de este tutorial: se empieza con la parte de actualización de información en función del resultado de la última acción realizada (en este caso, solo ver si se cayó en una casilla de tipo zapatillas), y después aparece la parte que desarrolla el comportamiento en sí (que va desde la línea 57 a la línea 77).

En la versión inicial del código, se incluyen algunos métodos para ayudar al estudiante a elaborar la práctica. Uno de esos métodos es ***ActualizarMapa***(). Esta función pone en las matrices **mapaResultado** y **mapaCotas** los valores que tienen los sensores de los agentes, de manera que por donde vayan pasando, se irá proyectando en el mapa 2D lo que sus sensores han ido captando. Una descripción de todas las funciones incorporadas se puede consultar en la sección ?.

La línea 55 indica que si el agente, en el instante actual se encuentra encima de una casilla de tipo zapatilla (recordar que la posición 0 del vector de visión es la información sobre la casilla en la que se encuentra el agente y que el vector `superficie` es el que indica los accidentes geográficos), se adquiere el objeto `zapatillas`.

Se puede observar que el comportamiento mantiene una estructura del tipo "`if-then-else`" tal y como ya se indicaba en la introducción. Esta estructura puede ser cómoda para definir, pero sobre todo para depurar el comportamiento.

```
46  // Niveles iniciales (Comportamientos reactivos simples)
47  Action ComportamientoIngeniero::ComportamientoIngenieroNivel_0(Sensores sensores)
48  {
49    Action accion = IDLE;
50    // El comportamiento de seguir un camino hasta encontra una planta de T. Residuos
51    // Poner el valor de los sensores de visión sobre los mapas.
52    ActualizarMapa(sensores);
53
54    // Actualización de variables de estado
55    if (sensores.superficie[0] == 'D') tiene_zapatillas = true;
56
57    //Definición del comportamiento
58    if (sensores.superficie[0] == 'U'){ // Llegué a una 'U'
59      return IDLE;
60    }
61    else if (sensores.superficie[2] == 'C'){
62      accion = WALK;
63    }
64    else if (sensores.superficie[1] == 'C'){
65      accion = TURN_SL;
66    }
67    else if (sensores.superficie[3] == 'C'){
68      accion = TURN_SR;
69    }
70    else {
71      accion = TURN_SL;
72    }
73
74    // Devolver la siguiente accion a hacer
75    last_action = accion;
76    return accion;
77  }
```

Una descripción rápida en lenguaje natural de las reglas que describen el comportamiento sería:

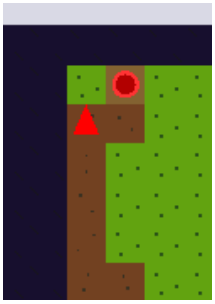
- Línea 58, si el agente está en una casilla de tipo `'U'` (planta T. Residuos) se ha cumplido el objetivo, por lo tanto, "no hagas nada".
- Línea 61, es la regla de avanzar si delante hay una casilla de tipo camino.
- Línea 64, es la regla de orientarse 45 grados a la izquierda porque se detecto en esa dirección una casilla de tipo camino.
- Línea 67, si hay una casilla de tipo camino en la parte derecha, se orienta para intentar avanzar por esa casilla.
- Por último, en la línea 70 se establece la regla por defecto que indica que si ninguna de las anteriores es cierta es porque no hay camino delante en cuyo caso gira hacía un lado para buscar casillas de tipo camino.

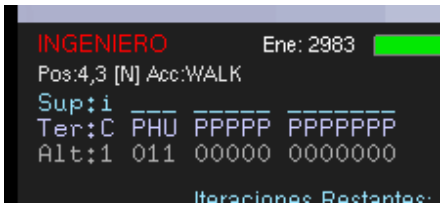
En la última parte se devuelve la acción seleccionada, pero antes se almacena en la variable de estado **`last_action`**, la acción que se va a enviar para su ejecución.

Ahora pasaremos a probar el funcionamiento de este comportamiento, para ello ejecutar desde un línea de comandos

./practica2 -m mapas/mapa30.map -n 0 -i 17 5 0 -t 17 17 0

Una vez levantado el entorno, en "***2D Map***" seleccionamos "***Ver***" para visualizar el mapa y pulsamos el botón de "***Paso***" para ver como el agente `Ingeniero` va avanzando por el camino.





Se puede observar que llega a la situación donde en su línea de visión, en la posición 3, hay una casilla `T. Residuos`, pero ignora orientarse hacia ella. De hecho, si se continúa con la simulación, se verá que nunca accede a una casilla que sea distinta de una casilla de tipo `camino`. La razón es simple, en el comportamiento no se considera acceder a casillas `T. Residuos`. Parece razonable que las casillas `T. Residuos` tengan prioridad sobre las casillas de tipo `camino`.

Con esa intención, pasamos a modificar el código creando la función ***`VeoCasillaInteresanteI`*** justo antes de ***`ComportamientoIngenieroNivel_0`***:

```
47  /**
48   * @brief Determina la mejor opcion entre las 3 casillas que tiene delante.
49   * @param i terreno que hay en la posición 1 de superficie (45 izq)
50   * @param c terreno que hay en la posición 2 de supeficie (justo delante)
51   * @param d terreno que hay en la posición 3 de supeficie (45 dch)
52   * @return 2 si es mejor WALK, 1 para TURN_SL y 3 para TURN_SR. 0 no hay nada interesante.
53   */
54  int VeoCasillaInteresanteI (char i, char c, char d){
55    if (c == 'U') return 2;
56    else if (i == 'U') return 1;
57    else if (d == 'U') return 3;
58    else if (c == 'C') return 2;
59    else if (i == 'C') return 1;
60    else if (d == 'C') return 3;
61    else return 0;
62  }
```

Devuelve 2, si la opción más interesante sería estar en la casilla de justo delante, 1 si sería la de 45 grados a la izquierda y 3 si fuera la de 45 grados a la derecha.

Modificamos ahora el comportamiento usando la función anterior, y quedando el método ***ComportamientoIngenieroNivel_0***.

```
64  // Niveles iniciales (Comportamientos reactivos simples)
65  Action ComportamientoIngeniero::ComportamientoIngenieroNivel_0(Sensores sensores)
66  {
67    Action accion = IDLE;
68    // El comportamiento de seguir un camino hasta encontra una planta de T. Residuos
69    // Poner el valor de los sensores de visión sobre los mapas.
70    ActualizarMapa(sensores);
71
72    // Actualización de variables de estado
73    if (sensores.superficie[0] == 'D') tiene_zapatillas = true;
74
75    //Definición del comportamiento
76    if (sensores.superficie[0] == 'U'){ // Llegué a una 'U'
77      return IDLE;
78    }
79
80    int pos = VeoCasillaInteresanteI(sensores.superficie[1], sensores.superficie[2], sensores.superficie[3]);
81    switch (pos)
82    {
83    case 2:
84      accion = WALK;
85      break;
86    case 1:
87      accion = TURN_SL;
88      break;
89    case 3:
90      accion = TURN_SR;
91      break;
92    default:
93      accion = TURN_SL;
94      break;
95    }
96
97    // Devolver la siguiente accion a hacer
98    last_action = accion;
99    return accion;
100 }
```

Podemos observar que en la línea 80 se invoca a la función ***VeoCasillaInteresanteI*** y en la variable `pos` se almacenará el valor 0, si no hay nada interesante, 1 si conviene avanzar por la de 45 grados a la izquierda, 2 si seguir recto o 3 si es mejor el avance por la de 45 grados a la derecha. En lo que en la versión inicial era una estructura condicional `if-then-else` sobre las tres opciones, ahora se ha transformado en un `switch` pero con la misma intención.

Si compilamos la nueva versión y volvemos a ejecutar

./practica2 -m mapas/mapa30.map -n 0 -i 17 5 0 -t 17 17 0

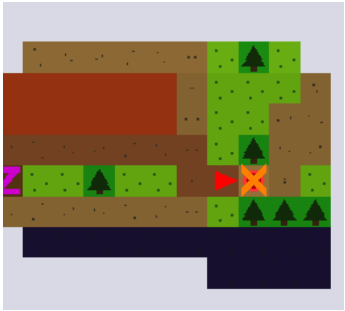
Veremos que ahora sí que el agente termina en la casilla de tipo `T. Residuos` que la vez anterior rechazó.

3. Considerando la altura

Probemos a ver que ocurre si ejecutamos la siguiente llamada:

./practica2 -m ./mapas/mapa30.map -n 0 -i 24 7 2 -t 17 17 0

Podemos ver que empieza a avanzar hacía la casilla 'U' situada en la parte inferior del mapa, pero que justo cuando va a acceder y tras realizar una acción de avance, el simulador manda un mensaje de texto (qué se puede ver en el terminal) diciendo *"[Ingeniero] Choco (casilla alta)"*.



Si en el simulador miramos lo que dice el sensor de cota del agente Ingeniero vemos que nos dice que la casilla que tiene justo delante está a altura 3 mientras que la casilla en la que se encuentra está a altura 1, es decir, hay un desnivel de 2 entre ambas casillas que solo sería superable si tuviera las `zapatillas`.

Debemos corregir el código en dos sentidos, por un lado, tener en cuenta la altura a la hora de tomar la decisión sobre cuál es la siguiente casilla interesante a visitar y, por otro lado, dotar al agente de la posibilidad de acceder a una casilla tipo `zapatillas` para conseguir ese objeto.

Para lo segundo, vamos a modificar la función ***VeoCasillaInteresanteI*** que es la que establece por donde queda la siguiente casilla más interesante a visitar. En este caso, parecería razonable que siga siendo la casilla `T. Residuos` la más interesante, pero si no tengo las `zapatillas` aún, poner por delante visitar una casilla de tipo `zapatillas` antes que una casilla de tipo `camino`. Cambio la parametrización de dicha función para incluir un parámetro de tipo `bool` que indicará si ya tengo las `zapatillas`. En el interior, altero la estructura de estructuras condicionales para

incluir la casilla de tipo zapatilla. El resultado final de todas esas modificaciones daría el siguiente código:

```
47  /**
48   * @brief Determina la mejor opcion entre las 3 casillas que tiene delante.
49   * @param i terreno que hay en la posición 1 de superficie (45 izq)
50   * @param c terreno que hay en la posición 2 de supeficie (justo delante)
51   * @param d terreno que hay en la posición 3 de supeficie (45 dch)
52   * @param zap indica si estoy en posesión de las zapatillas
53   * @return 2 si es mejor WALK, 1 para TURN_SL y 3 para TURN_SR. 0 no hay nada interesante.
54   */
55  int VeoCasillaInteresanteI (char i, char c, char d, bool zap){
56    if (c == 'U') return 2;
57    else if (i == 'U') return 1;
58    else if (d == 'U') return 3;
59    else if (!zap){
60      if (c == 'D') return 2;
61      else if (i == 'D') return 1;
62      else if (d == 'D') return 3;
63    }
64    if (c == 'C') return 2;
65    else if (i == 'C') return 1;
66    else if (d == 'C') return 3;
67    else return 0;
68  }
```

Si en ***CompotarmientoIngenieroNivel_0*** cambiamos la invocación a la función de ***VeoCasillaInteresanteI*** por:

```
int pos = VeoCasillaInteresanteR(sensores.superficie[1], sensores.superficie[2],
                                 sensores.superficie[3], tiene_zapatillas);
```

y volvemos a compilar, y ejecutamos

./practica2 -m ./mapas/mapa30.map -n 0 -i 24 7 2 -t 17 17 0

Vemos que ahora sí que el agente `Ingeniero` llega a la casilla puesto base tras coger las zapatillas.

Sin embargo, es posible que nos encontremos en una situación donde no tengamos las zapatillas y haya una diferencia de altura que nos impida el paso, por tanto, es fundamental el considerar la diferencia de altura en los movimientos de los agentes. Probar que el agente se queda atascado colisionando constantemente si ejecutamos:

./practica2 -m ./mapas/mapa30.map -n 0 -i 24 10 2 -t 17 17 0

Para solucionarlo, modificamos el comportamiento del agente en el siguiente sentido: el agente se orientará para avanzar hacía la casilla más interesante que él pueda alcanzar haciendo una acción `WALK`, es decir, aquella que es adyacente a donde se encuentra y es accesible por altura al agente.

Para ello, se modificará el código para comprobar las diferencias de altura entre la casilla actual del agente y las posiciones 1, 2 y 3 de su vector cota antes de invocar a la función ***`VeoCasillaInteresanteI`***, de manera que los argumentos que tengan una diferencia de altura que no permita el movimiento se pasarán como si fueran precipicios (se usará 'P' pero valdría cualquier tipo de terreno que no sea ni `'C'`, ni `'D'`, ni `'X'`). Para esta tarea, haré uso de la función auxiliar ***ViablePorAlturaI*** :

```
/**
 * @brief Determina si casilla viable por altura.
 * @param casilla tipo de terreno
 * @param dif diferencia de altura entre casillas
 * @param zap indica si estoy en posesión de las zapatillas
 * @return 'P' si no es accesible por altura y casilla en otro caso
 */
char ViablePorAlturaI (char casilla, int dif, bool zap){
  if (abs(dif) <= 1 or (zap and abs(dif) <=2))
    return casilla;
  else
    return 'P';
}
```

Ahora la función `ComportamientoIngenieroNivel_0` quedaría como muestra en el siguiente código.

```
83  // Niveles iniciales (Comportamientos reactivos simples)
84  Action ComportamientoIngeniero::ComportamientoIngenieroNivel_0(Sensores sensores)
85  {
86    Action accion = IDLE;
87    // El comportamiento de seguir un camino hasta encontra una planta de T. Residuos
88    // Poner el valor de los sensores de visión sobre los mapas.
89    ActualizarMapa(sensores);
90
91    // Actualización de variables de estado
92    if (sensores.superficie[0] == 'D') tiene_zapatillas = true;
93
94    //Definición del comportamiento
95    if (sensores.superficie[0] == 'U'){ // Llegué a una 'U'
96      return IDLE;
97    }
98
99    char i = ViablePorAlturaI(sensores.superficie[1], sensores.cota[1]-sensores.cota[0], tiene_zapatillas);
100   char c = ViablePorAlturaI(sensores.superficie[2], sensores.cota[2]-sensores.cota[0], tiene_zapatillas);
101   char d = ViablePorAlturaI(sensores.superficie[3], sensores.cota[3]-sensores.cota[0], tiene_zapatillas);
102
103   int pos = VeoCasillaInteresanteI(i, c, d, tiene_zapatillas);
104
105   switch (pos)
106   {
107   case 2:
108     accion = WALK;
109     break;
110   case 1:
111     accion = TURN_SL;
112     break;
113   case 3:
114     accion = TURN_SR;
115     break;
116   default:
117     accion = TURN_SL;
118     break;
119   }
120
121   // Devolver la siguiente accion a hacer
122   last_action = accion;
123   return accion;
124 }
```

En las líneas de la 99 a la 101 se invoca a la función ***ViablePorAlturaI*** para asignar a las variables `i` (izquierda), `c` (centro), `d` (derecha) el carácter asociado a la casilla que se ve en el sensor de superficie siempre que dicha casilla sea accesible por altura. A partir de esas 3 variables locales, `i`, `c` y `d`, se determina la más interesante invocando ahora a ***VeoCasillaInteresanteI***.

Si compilamos y volvemos a ejecutar

./practica2 -m ./mapas/mapa30.map -n 0 -i 24 10 2 -t 17 17 0

Observamos que ahora ya no se queda bloqueado en la colisión con la casilla, sino que se sigue moviendo buscando una nueva forma para llegar a una casilla `T. Residuos`.

4. Replicando el comportamiento para el agente Técnico

Esta sección se plantea como una tarea a realizar por el estudiante para replicar lo que se ha hecho en este tutorial con el agente Ingeniero. Recordaremos los pasos a seguir:

1. Definir las variables de estado necesarias. Dicha definición se realizará en la parte privada de la clase `ComportamientoTécnico` en el fichero "**tecnico.hpp**". Para este agente será necesario definir las mismas 3 variables de estado: `last_action` y `tiene_zapatillas`, y además se inicializarán de la misma forma que se hizo con el otro agente.

2. Se han de definir las funciones:
   (a) `VeoCasillaInteresanteT`
   (b) `ViablePorAlturaT`

   IMPORTANTE:

   I. En las funciones (que no sean métodos de la clase) añadir al final del identificador de la función un `I` para las que sean para el agente `Ingeniero` y una `T` para las del agente `Técnico`, para evitar que el compilador nos diga que estamos redefiniendo la función.

   II. Si el estudiante lo desea, estas dos funciones podrían ser implementadas, no como funciones auxiliares, sino como funciones miembro de la clase. En este segundo caso, ya no sería necesario añadir al final del identificador las letras I o T.

   III. En este nivel, para el agente `Técnico` no es relevante tener zapatillas, ya que no le permite moverse mejor por las casillas `camino`. Este hecho implicaría no considerar el parámetro `bool zap` en las dos funciones anteriores.

3. Se tiene que repetir el esquema del comportamiento definido en ***`ComportamientoTécnicoNivel_0`***. Hay que tener en cuenta que el límite de diferencia de altura entre casillas que puede superar el `Técnico` es de 1 en valor absoluto en todos los casos.

Una vez completado y adaptado el comportamiento a las características especiales del agente `Técnico` probar que funciona correctamente con la siguiente llamada

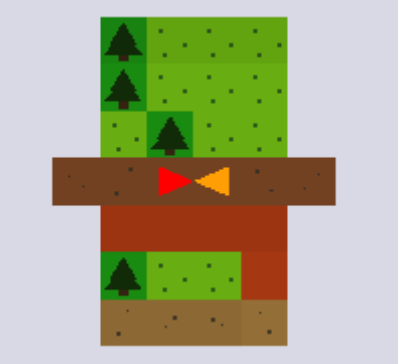
./practica2 -m ./mapas/mapa30.map -n 0 -i 24 10 2 -t 17 17 0

5. Resolviendo las interacciones entre los agentes

Terminada la tarea propuesta en la sección anterior, el estudiante tiene definido un comportamiento para cada uno de los agentes que aparentemente resuelve lo que se pide en el nivel. Pero veamos que ocurre en una ejecución como:

./practica2 -m ./mapas/mapa30.map -n 0 -i 16 9 2 -t 16 14 6

Se puede observar que cada agente bloquea a otro impidiendo que desarrollen su comportamiento de forma normal. ¿Cómo podría resolverse interacciones de este tipo?



Es evidente que en la definición de los comportamientos de los agentes no se ha tenido en cuenta la posibilidad que los agentes se pudieran encontrar y resulta muy evidente que es necesario que esto se contemple.

En este punto quiero recordar que el orden usado por el simulador para aplicar las acciones de los agentes es: primero al agente `Ingeniero` y después la acción del agente `Técnico`. Tener en cuenta este orden puede ayudar a resolver situaciones de conflicto, ya que el agente `Ingeniero` siempre puede evitar la colisión, pero el `Técnico` no.

También se propone en este tutorial como tarea que el alumno sea capaz de evitar este tipo de interacciones entre los agentes en el nivel 0. Uno de los test de la autoevaluación irá expresamente dirigido a comprobar que el estudiante ha diseñado un comportamiento para resolver este problema.

6. Funciones Incluidas en el Código

Con la intención de ayudar al estudiante en la realización de la práctica y ahorrarle dolores de cabeza con el manejo de las orientaciones y las posiciones relativas entre sensores y las variables **mapaResultado** y **mapaCotas**, se incluyen en el software base algunos métodos ya implementados de la clase ***ComportamientoIngeniero***.

Estos métodos son los siguientes:

```
/**
* @brief Actualiza la información del mapa interno basándose en los sensores.
* Actualiza mapaResultado y mapaCotas
* con la información de los 16 sensores (casilla actual + 15 casillas alrededor).
*/
void ActualizarMapa(Sensores sensores);
```

```
/**
* @brief Comprueba si una casilla es transitable.
* @param f Fila de la casilla.
* @param c Columna de la casilla.
* @param tieneZapatillas Indica si el agente posee zapatillas.
* @return true si la casilla es transitable (no es muro ni precipicio).
*/
bool EsCasillaTransitableLevel0(int f, int c, bool tieneZapatillas);
```

```
/**
* @brief Comprueba si la casilla de delante es accesible por diferencia de altura.
* REGLAS: Desnivel máximo 1 sin zapatillas, 2 con zapatillas.
* @param actual Estado actual del agente (fila, columna, orientacion).
* @return true si el desnivel con la casilla de delante es admisible.
*/
bool EsAccesiblePorAltura(const ubicacion &actual, bool zap);
```

```
/**
*Tipo de Dato ubicación: almacena una posición de un agente con orientación, es decir, su fila (f),
* su columna (c) y su orientación (brujula).
*/
struct ubicacion {
int f;
int c;
Orientacion brujula;

bool operator==(const ubicacion &ub) const {
return (f == ub.f and c == ub.c and brujula == ub.brujula);
}
};
```

```
/**
* @brief Devuelve la posición (fila, columna) de la casilla que hay delante del agente.
* @param actual Estado actual del agente (fila, columna, orientacion).
* @return Estado con la fila y columna de la casilla de enfrente.
*/
ubicacion Delante(const ubicacion &actual) const;
```

```
/**
* @brief Imprime por consola la secuencia de acciones de un plan para un agente.
* @param plan Lista de acciones del plan.
*/
void PintaPlan(const list<Action> &plan);
```

```
/**
* Tipo de Dato que se usa para definir la planificación de la red de tuberías.
* Cada paso del plan almacena la fila (fil), columna (col) y la operación que hay que hacer sobre
* el terrero (op), significando -1 una acción de DIG, 1 una acción de RAISE y 0 ninguna operación
*/
struct Paso {
int fil;
int col;
int op; // -1 bajar la casilla, 0 dejar como está, 1 subir la casilla
};
```

```
/**
* @brief Imprime las coordenadas y operaciones de un plan de tubería.
* @param plan Lista de pasos (fila, columna, operación).
*/
void PintaPlan(const list<Paso> &plan);
```

```
/**
* @brief Convierte un plan de acciones en una lista de casillas para
* su visualización en el mapa gráfico.
* @param st Estado de partida.
* @param plan Lista de acciones del plan.
*/
void VisualizaPlan(const ubicacion &st, const list<Action> &plan);
```

```
/**
* @brief Convierte un plan de tubería en la lista de casillas usada
* por el sistema de visualización.
* @param st Estado de partida (no utilizado directamente).
* @param plan Lista de pasos del plan de tubería.
*/
```

```
void VisualizaRedTuberias(const list<Paso> &plan);
```

Los métodos anteriores están definidos en el fichero del ingeniero. Los métodos definidos para el técnico son los mismos exceptuando que no están implementados (porque en principio no hace falta) los relacionados con la tuberías, ya que es ingeniero el encargado de gestionar directamente ese plan.

7. Comentarios Finales

Este tutorial tiene como objetivo dar un pequeño empujón en el inicio del desarrollo de la práctica y lo que se propone es solo una forma de dar respuesta (la más básica en todos los casos) a los problemas con los que os tenéis que enfrentar. Por tanto, todo lo que se propone aquí es mejorable y lo debéis mejorar.

Los dos primeros niveles de la práctica trata sobre agentes reactivos, por consiguiente, los comportamientos deliberativos no están permitidos.

Muchos elementos que forman parte de la práctica no se han tratado en este tutorial. Esos elementos son relevantes para mejorar la capacidad de exploración del agente e instamos a que se les preste atención.

Por último, resaltar que la práctica es individual y que la detección de copias (trozos de código iguales o muy parecidos entre estudiantes y que el estudiante no sepa explicar su funcionamiento) implicará el suspenso en la asignatura.